

WEEK 6 TASK

Name: Rohit Balasaheb Kokare

ID: CT_CSI_DE_1172

College: Sanjivani College of Engineering

Objective:

Objective: Understand Spark architecture and perform efficient data processing using transformations, filtering, schema handling, and optimized file formats. Steps: Understand Spark architecture (Driver, Cluster Manager, Executors) and execution modes. Learn Lazy Evaluation and how it optimizes execution using DAG (Lineage Graph). Read data from files (CSV, Parquet) with proper schema handling. Perform filtering and selection of required columns. Modify DataFrames (rename columns, cast data types, add new columns). Apply transformations and actions appropriately. Understand wide transformations and performance concepts (Shuffle, Predicate Pushdown). Work with different file formats (CSV vs Parquet) and their impact on performance. Handle null values and filter datasets efficiently. Build data pipelines (read → transform → filter → write). Save processed data into required formats (CSV/Parquet). Follow best practices for large datasets (avoid collect(), use show()). Output: Spark code (PySpark/Scala) + execution results + brief insights on performance and architecture.

Resources:

https://celebaltech.sharepoint.com/:w:/s/Celebal-LMS/IQDLvM3UIKsvSLKuDW6nbDK9AaeiXu7C2zJOow_bWTU6WNo?e=nwlp8h

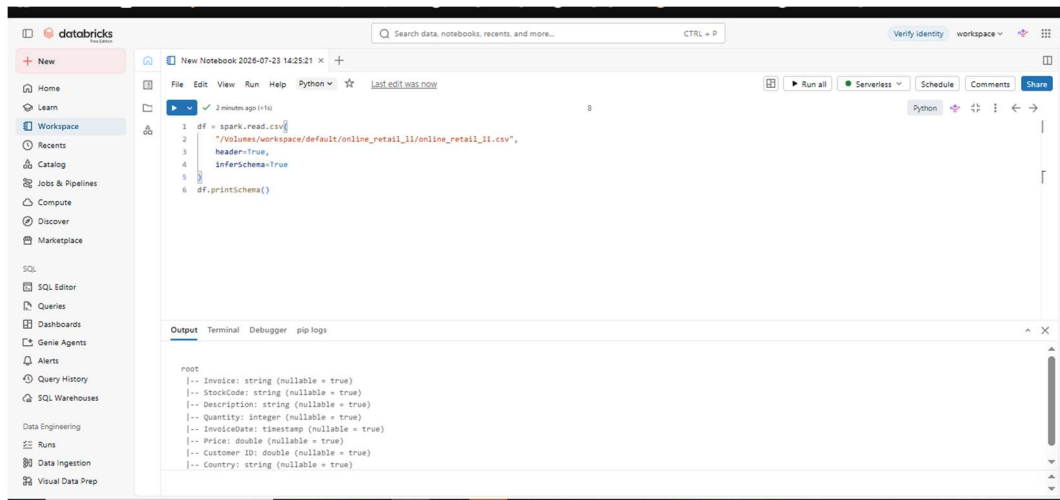
Step 1: Import Required Libraries

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

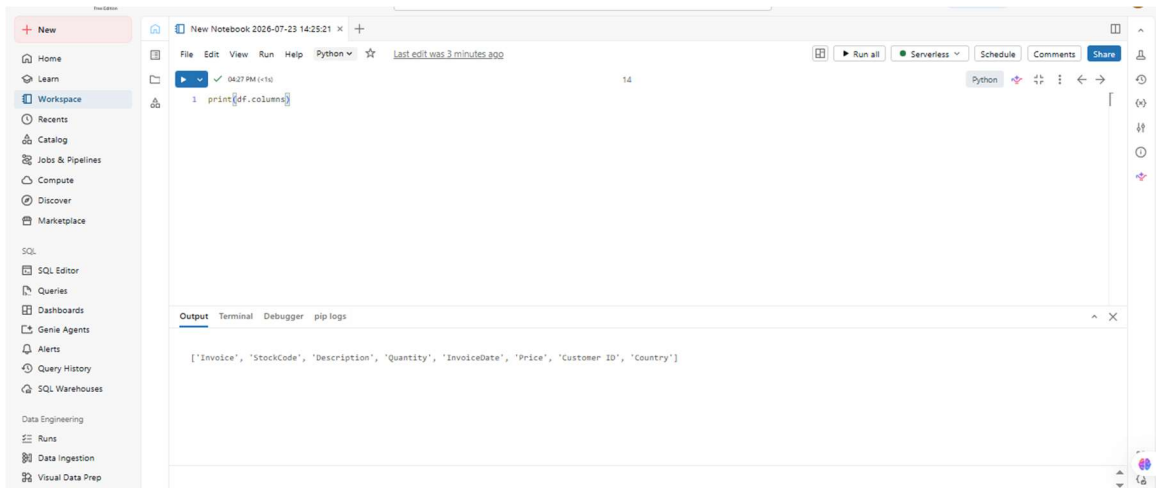
Step 2: Read CSV File

```
df = spark.read.csv(
    "/Volumes/workspace/default/online_retail_11/online_retail_II.csv",
    header=True,
    inferSchema=True
)
df.printSchema()
```

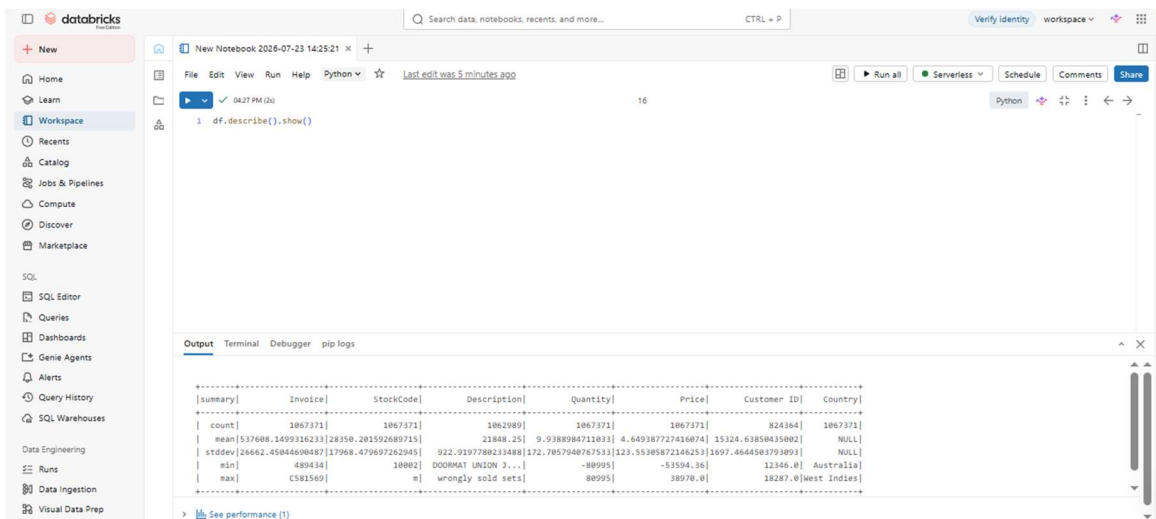
Output:



Step 3: Display Columns



Step 4: Summary Statistics



Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Driver

- Creates the SparkSession.
- Converts user code into tasks.
- Builds the DAG (Directed Acyclic Graph).
- Schedules tasks and collects results.

Cluster Manager

- Allocates CPU, memory, and other resources.
- Manages executors.
- Examples: Standalone, YARN, Kubernetes.

Executors

- Execute tasks assigned by the Driver.
- Process data in parallel.
- Cache data and return results to the Driver.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer:

Spark records all transformations without executing them immediately. These transformations form a Lineage Graph (DAG). Execution begins only when an Action such as `show()` or `count()` is called.

Advantages

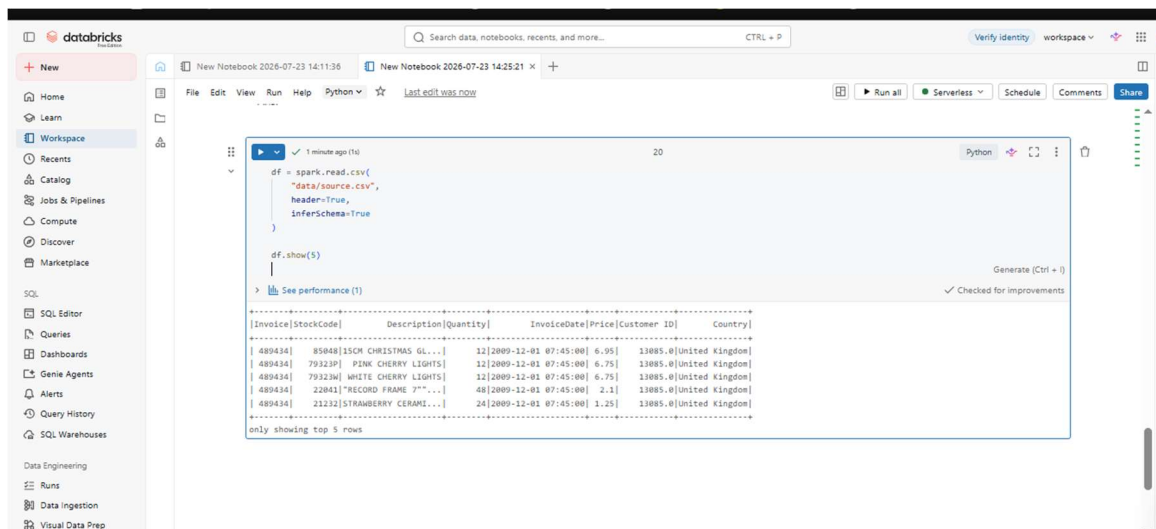
- Eliminates unnecessary computations.
- Optimizes execution plans.
- Reduces disk I/O.
- Improves performance.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv(
    "data/source.csv",
    header=True,
    inferSchema=True
)

df.show(5)
```

output:



Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV

Row-based
Large file size
Slower
Reads all columns
No Predicate Pushdown

Parquet

Column-based
Compressed
Faster
Reads required columns only
Supports Predicate Pushdown

Why?

parquet improves performance because Spark reads only the required columns.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

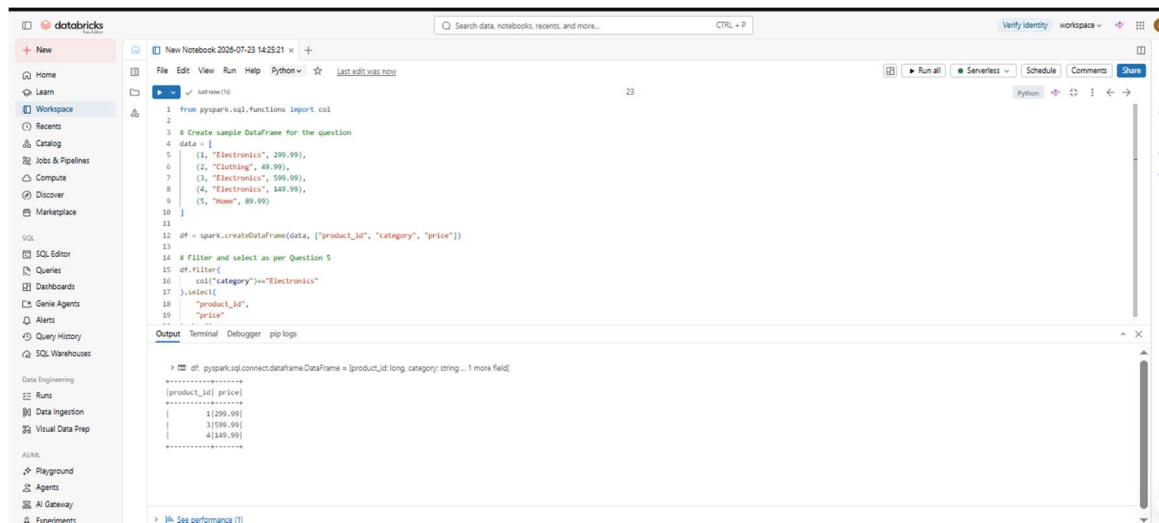
```
from pyspark.sql.functions import col

# Create sample DataFrame for the question
data = [
    (1, "Electronics", 299.99),
    (2, "Clothing", 49.99),
    (3, "Electronics", 599.99),
    (4, "Electronics", 149.99),
    (5, "Home", 89.99)
]

df = spark.createDataFrame(data, ["product_id", "category", "price"])

# Filter and select as per Question 5
df.filter(
    col("category")=="Electronics"
).select(
    "product_id",
    "price"
).show()
```

Output:



The screenshot shows a Databricks workspace interface. On the left is a sidebar with navigation options like Home, Workspace, Recent, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, and Performance. The main area displays a Python notebook with the code from the previous block. The output of the final `show()` command is visible in the bottom panel, showing a table with two columns: `product_id` and `price`. The table contains three rows of data corresponding to the 'Electronics' category.

product_id	price
1	299.99
3	599.99
4	149.99

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

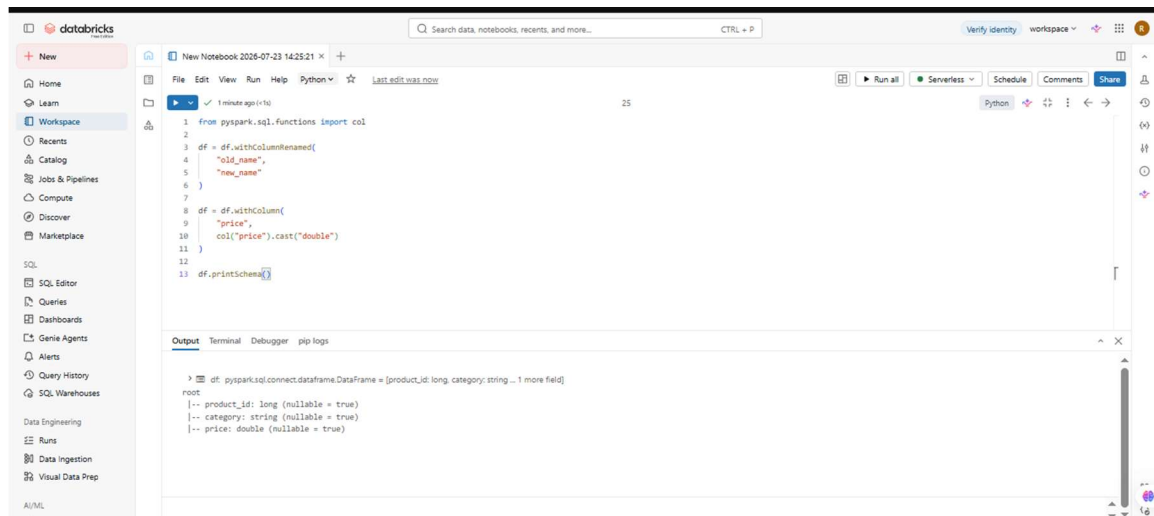
```
from pyspark.sql.functions import col

df = df.withColumnRenamed(
    "old_name",
    "new_name"
)

df = df.withColumn(
    "price",
    col("price").cast("double")
)

df.printSchema()
```

Output:



The screenshot shows a Databricks workspace interface. On the left is a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main area displays a Python notebook titled 'New Notebook 2026-07-23 142521'. The code in the notebook is as follows:

```
1 from pyspark.sql.functions import col
2
3 df = df.withColumnRenamed(
4     "old_name",
5     "new_name"
6 )
7
8 df = df.withColumn(
9     "price",
10    col("price").cast("double")
11 )
12
13 df.printSchema()
```

Below the code editor, the 'Output' tab is selected, showing the schema output:

```
> dt: pyspark.sql.connect.dataframe.DataFrame = [product_id: long, category: string ... 1 more field]
root
 |-- product_id: long (nullable = true)
 |-- category: string (nullable = true)
 |-- price: double (nullable = true)
```

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Spark stores every transformation in a **Lineage Graph (DAG)**.

If an executor fails, Spark does not reload the entire dataset. It recomputes only the lost partitions using the lineage information.

Benefits:

- Automatic recovery
- Fault tolerance
- Efficient recomputation

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

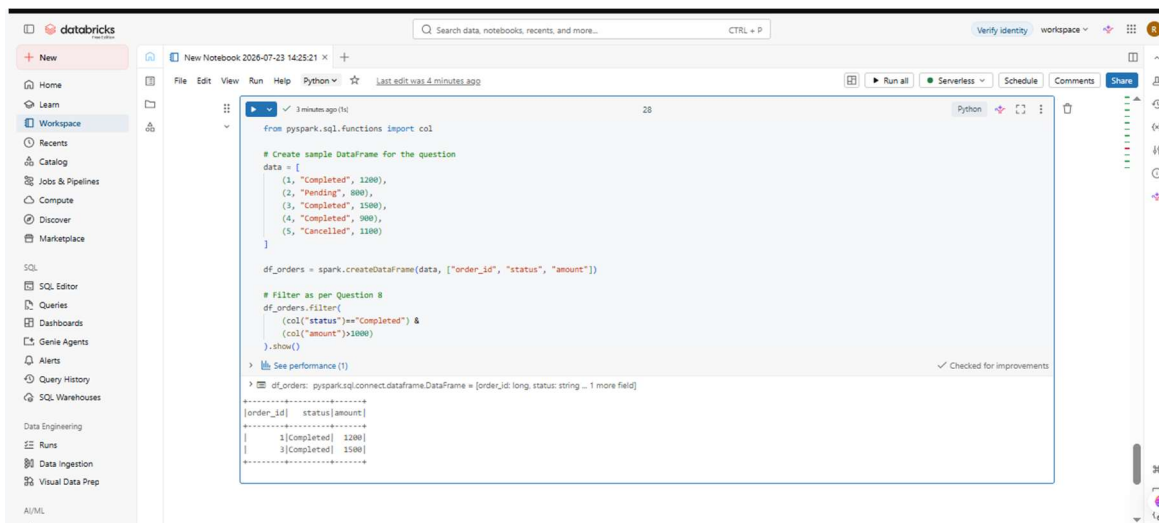
```
from pyspark.sql.functions import col

# Create sample DataFrame for the question
data = [
    (1, "Completed", 1200),
    (2, "Pending", 800),
    (3, "Completed", 1500),
    (4, "Completed", 900),
    (5, "Cancelled", 1100)
]

df_orders = spark.createDataFrame(data, ["order_id", "status", "amount"])

# Filter as per Question 8
df_orders.filter(
    (col("status")== "Completed") &
    (col("amount")>1000)
).show()
```

Output:



Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer:

Predicate Pushdown pushes filter conditions to the storage layer (such as Parquet).

Example:

Instead of reading all rows,

Spark reads only matching rows.

Benefits:

- Faster execution
- Lower memory usage
- Less disk I/O

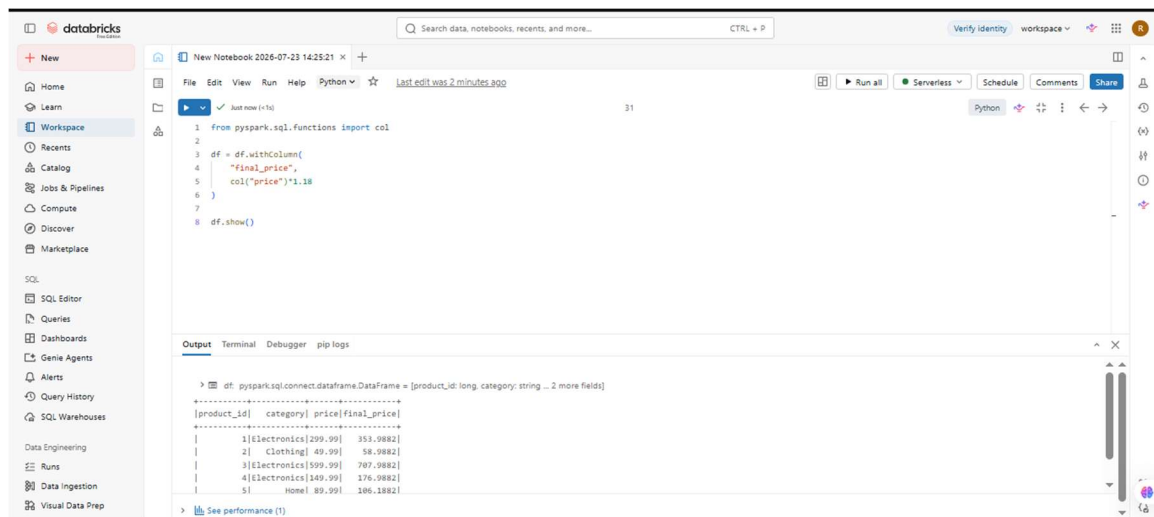
Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col

df = df.withColumn(
    "final_price",
    col("price")*1.18
)

df.show()
```

Output:



Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Transformations	Actions
Lazy	Immediate execution
Return DataFrame	Return result
Build DAG	Execute DAG

. Examples of Transformations

```
df.filter(col("price")>100)
```

```
df.select("product")
```

Examples of `Actions`

```
df.show()
```

```
df.count()
```

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
from pyspark.sql.functions import col

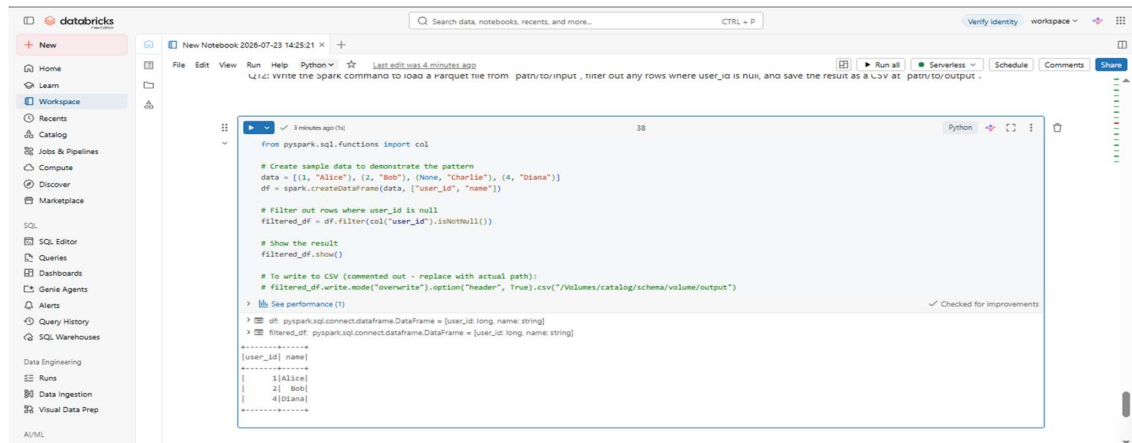
# Create sample data to demonstrate the pattern
data = [(1, "Alice"), (2, "Bob"), (None, "Charlie"), (4, "Diana")]
df = spark.createDataFrame(data, ["user_id", "name"])

# Filter out rows where user_id is null
filtered_df = df.filter(col("user_id").isNotNull())

# Show the result
filtered_df.show()

# To write to CSV (commented out - replace with actual path):
# filtered_df.write.mode("overwrite").option("header",
# True).csv("/Volumes/catalog/schema/volume/output")
```

Output:



The screenshot shows a Databricks workspace with a notebook titled 'New Notebook 2026-07-23 14:25:21'. The notebook contains a Python script that creates sample data, filters out rows where user_id is null, and shows the result. The output of the script is displayed below the code, showing a table with two columns: user_id and name. The data rows are: 1 | Alice, 2 | Bob, 4 | Diana.

```
from pyspark.sql.functions import col

# Create sample data to demonstrate the pattern
data = [(1, "Alice"), (2, "Bob"), (None, "Charlie"), (4, "Diana")]
df = spark.createDataFrame(data, ["user_id", "name"])

# Filter out rows where user_id is null
filtered_df = df.filter(col("user_id").isNotNull())

# Show the result
filtered_df.show()

# To write to CSV (commented out - replace with actual path):
# filtered_df.write.mode("overwrite").option("header", True).csv("/Volumes/catalog/schema/volume/output")

> See performance (1)
```

user_id	name
1	Alice
2	Bob
4	Diana

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Client Mode	Cluster Mode
Driver runs on local machine	Driver runs inside cluster
Development	Production
Client must remain connected	Runs independently

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
from pyspark.sql.functions import col

# Create sample DataFrame for the question
data = [
    (1, "North", "Medium"),
    (2, "South", "High"),
    (3, "North", "Low"),
    (4, "East", "Medium"),
    (5, "West", "High")
]
```

```
df_filter = spark.createDataFrame(data, ["id", "region", "priority"])

# Filter as per Question 14: region is 'North' OR priority is 'High'
df_filter.filter(
    (col("region")== "North") |
    (col("priority")== "High")
).show()
```

Output:

The screenshot shows the Databricks interface. The notebook code is as follows:

```
1 from pyspark.sql.functions import col
2
3 # Create sample DataFrame for the question
4 data = [
5     (1, "North", "Medium"),
6     (2, "South", "High"),
7     (3, "North", "Low"),
8     (4, "East", "Medium"),
9     (5, "West", "High")
10 ]
11
12 df_filter = spark.createDataFrame(data, ["id", "region", "priority"])
13
14 # Filter as per Question 14: region is 'North' OR priority is 'High'
15 df_filter.filter(
16     (col("region")== "North") |
17     (col("priority")== "High")
18 ).show()
```

The output of the code is displayed in the bottom panel:

```
> df_filter: pyspark.sql.connect.dataframe.DataFrame = [id: long, region: string ... 1 more field]
-----
| id|region|priority|
-----
| 1| North|  Medium|
| 2| South|   High|
| 3| North|   Low|
| 5| West|   High|
-----
```

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer:

`show(5)` returns only five rows.

`collect()` returns the entire dataset to the Driver.

For large datasets:

- collect() may cause OutOfMemoryError.
- Slows execution.
- Can crash the Driver.

Therefore, show(5) is safer for data exploration.

Brief Insights:

1. Spark Architecture enables distributed processing using the Driver, Cluster Manager, and Executors for efficient execution.
2. Lazy Evaluation and DAG optimize query execution by delaying computation until an action is performed.
3. DataFrame transformations, filtering, schema handling, and null value processing improve data quality and efficiency.
4. Parquet provides better performance than CSV due to its columnar storage, compression, and Predicate Pushdown.
5. Building an ETL pipeline (Read → Transform → Filter → Write) demonstrates efficient data processing using PySpark.

Conclusion:

This assignment provided practical experience with Apache Spark and PySpark for processing large datasets efficiently. It enhanced my understanding of Spark Architecture, Lazy Evaluation, DAG, DataFrame operations, schema handling, filtering, transformations, and null value management.

I also learned the advantages of Parquet over CSV, the importance of Shuffle and Predicate Pushdown in performance optimization, and how to build a complete ETL pipeline (Read → Transform → Filter → Write). Overall, this assignment strengthened my practical knowledge of PySpark and improved my ability to develop scalable and efficient data processing solutions using Apache Spark.